

Preregistered Paired Evaluation of a Deterministic Verifier Pipeline for LLM-Generated Network Configuration Changes — Non-informative Result under Shared-Generation Semantics

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a fully preregistered paired experiment (cand-2026-01-prereg-v1) that tested whether a deterministic verifier pipeline (generate → deterministic_netops_validator_v5 → optional repair) reduces the rate of verifier-detected unsafe intent→configuration proposals produced by a hosted LLM on synthetic NetOps tasks. Design: N=200 preregistered unique intent→configuration tasks in a paired design comparing (a) baseline LLM-only and (b) guarded pipeline (gpt-5-mini + deterministic_netops_validator_v5). Execution used shared_initial_candidate generation semantics (one LLM call per task) producing model_calls_used = 200 (preregistered independent per-arm generation would have used up to 400). Observed (adversarial cluster): baseline SAFE = 200/200; guarded SAFE = 200/200 (paired contingency n11 = 200, n10 = 0, n01 = 0). Estimated paired SAFE-rate difference = 0.0; exact McNemar two-sided $p = 1.0$; paired bootstrap 95% CI = [0.0, 0.0] (resamples = 10000, seed = 1729). Because identical per-pair generations were produced and the observed baseline unsafe rate was 0%, the preregistered causal hypothesis (that the deterministic verifier reduces unsafe proposals) was not testable in this execution. We publish the complete preregistered run and artifacts, provide an artifact-grounded diagnosis of testability failure, and give concrete, archive-supported recommendations (independent per-arm generation budgeting, adversarial injection calibration, and exploratory ROC reserves) for informative repeats. All execution and analysis artifacts cited here are archived in the supplied bundle.

I. INTRODUCTION

Automating human-intent → device-configuration translation with large language models (LLMs) is an active NetOps research thrust because it can reduce human effort and speed maintenance tasks. Yet incorrectly specified or misordered changes can cause outages, policy violations, or security exposures. A commonly proposed mitigation is tool-augmentation: couple an LLM generator to deterministic validators, sandboxed simulators, or constrained executors in a generate-validate-repair pipeline so that proposed changes are checked before execution.

We preregistered and executed a confirmatory paired experiment (cand-2026-01-prereg-v1) to quantify whether deterministic verifier augmentation causally reduces the frequency of verifier-detected unsafe proposals from a hosted LLM under both benign and adversarial prompt clusters. The preregistered estimand was the paired SAFE-rate difference (guarded minus

baseline), tested by an exact McNemar two-sided test with a paired bootstrap CI (10000 resamples, seed = 1729). The design sampled N=200 tasks using a deterministic generator and a paired comparison across two pipelines. Two generation semantics were permitted in the preregistration: independent per-arm generation (one LLM call per arm per task) or shared_initial_candidate (one shared LLM call per task scored by both arms). The executed run used shared_initial_candidate to conserve model-call budget.

Key outcome: under the executed shared semantics the sampled LLM outputs were scored SAFE by the deterministic validator in every confirmatory episode (baseline SAFE = 200/200; guarded SAFE = 200/200). This concordant ceiling outcome (n11 = 200; n10 = n01 = n00 = 0) yields an estimate of zero paired difference and an exact McNemar $p = 1.0$. Because the observed baseline unsafe rate was 0% and the same candidate was used in both arms, the preregistered causal hypothesis could not be meaningfully evaluated. The manuscript therefore focuses on three contributions grounded in archived artifacts: (1) a complete, deterministic preregistered run published as reproducible artifacts; (2) a precise, artifact-grounded diagnosis of why this execution was non-informative for the confirmatory estimand; and (3) concrete, artifact-supported design recommendations that would enable informative repeats under the same preregistration framework.

II. RELATED WORK

Recent surveys and prototypes document strong interest in applying LLMs to NetOps tasks—intent translation, configuration synthesis, diagnosis, and limited remediation—but emphasize that demonstrated gains are largely in prototype or curated-case settings rather than in production deployments [openalex:W7161354925; openalex:W7170714602; openalex:W4411109869]. Several lines of prior work are directly relevant to our design choices and to the generate-validate-repair architecture we evaluated. First, multiple studies and proposals show that augmenting LLMs with deterministic tools (verifiers, simulators, or formal checkers) reduces certain classes of constraint violations on curated tasks: generate-validate-refine or divide-verify-refine pipelines measurably improve adherence to structured in-

structions when the verifier faithfully captures the task constraints [openalex:W4412888330; openalex:W7161354925]. These works, however, also report that improvement magnitude varies with verifier fidelity and the nature of the constraints being checked.

Second, research on guardrails, tool schemas, and MCP-style governance highlights practical trade-offs encountered by tool-augmented agents: stricter schema enforcement reduces some unsafe outputs but can increase false refusals or operator workload unless tuned carefully [TechRxiv:177006109; TechRxiv:177004277]. Red-teaming and guardrail-distillation literature from adjacent domains shows that adversarial prompts can often be engineered to bypass naive guardrails, motivating replayable, sandboxed evaluations and explicit ROC-style diagnostics for verifiers [openalex:W7172000952; TechRxiv:177006109].

Third, evaluation critiques argue that static benchmarks are insufficient for agentic NetOps: meaningful assessment requires workflow-centred, deterministic replay (sandbox) environments, archived execution traces, and paired causal designs that can separate generator variability from validator effects [openalex:W7161354925; doi:10.2139/ssrn.7268438]. These methodological prescriptions informed our preregistration (paired design, deterministic task generator, and archived validator traces) because they make it possible to ascribe observed outcome differences to pipeline interventions rather than to uncontrolled environmental variability.

Finally, prototype and architecture surveys stress operational integration concerns—human-in-the-loop gates, auditability, multi-tool orchestration, and simulator fidelity—all of which constrain what a verifier can safely assert in practice and how empirical evaluation results should be interpreted for operational readiness [TechRxiv:177004277; openalex:W4406549441]. In sum, prior work supports the conceptual value of verifier augmentation but also warns that empirical gains are task- and verifier-dependent and that evaluation semantics (for example, shared vs independent generation and verifier completeness) strongly affect the interpretability of paired causal contrasts. Our preregistered design attempted to operationalize these recommendations; the archived run and the diagnosis below show how a specific execution semantics (shared_initial_candidate) interacted with task difficulty and verifier outputs to produce a non-informative confirmatory outcome.

III. METHODOLOGY

Preregistration and estimand. The experiment was preregistered as cand-2026-01-prereg-v1. Primary estimand: paired_success_rate_difference_guarded_minus_baseline (guarded - baseline) on binary SAFE indicators obtained from a deterministic sandbox validator. Confirmatory test: exact two-sided McNemar implemented by paired_binary_analysis_v1; confidence intervals: paired bootstrap percentile (10000 resamples, seed = 1729). The preregistration specified a paired binary design with N=200 tasks, planned paired episodes = 400, and a power plan

that targeted detecting an approximate 7 percentage-point reduction in unsafe proposals with 80% power assuming a baseline unsafe rate $\approx 10\text{--}15\%$.

Task generator, scope, seeds, and adversarial design. Tasks were produced by a deterministic generator (deterministic_netops_task_generator_v5) seeded deterministically; per-task generator seeds were assigned sequentially (generation_seed 1..200) and the generator family seed was preregistered. Each task manifest includes: (i) an initial_state and expected_final_state; (ii) an explicit intent text; (iii) a required_sequence (ordered field changes) encoding workflow constraints; and (iv) machine-readable difficulty metadata (changed_interface_count, dependency_rule_count, required_command_count). The confirmatory set included a preregistered adversarial cluster generated by applying templated obfuscation transforms (negation/implication patterns, privilege-escalation phrasings, and contradiction constructs) to otherwise valid intents. All generator code, task manifests, and seeds are archived in execution/task_manifest.jsonl (artifact in bundle). Representative task manifest entries are present under execution/ (see artifact examples in the archived bundle). The deterministic task design ensured that tasks are replayable and that task difficulty metadata are reproducible.

Pipelines compared and validator semantics. We compared two pipelines paired per task: (1) baseline — accept the LLM candidate and score it post hoc with deterministic_netops_validator_v5; (2) guarded — subject the same candidate to deterministic_netops_validator_v5 in a sandbox and (optionally) allow a single automatic repair call per preregistered rules. For the confirmatory episodes no repairs were applied (repair_* fields are null in score JSONs), so guarded outcomes reflect deterministic validation verdicts alone. The validator applies full_intent_constraint_validation and emits a deterministic boolean validator.valid and a structured violations list; the binary confirmatory outcome uses validator.valid (1 = SAFE, 0 = UNSAFE). Representative validator traces are archived in execution/scoring/*.json (see examples execution/scoring/task-000001-baseline.json and execution/scoring/task-000001-guarded.json).

Generation semantics, model budgeting, and executed choices. The preregistration explicitly allowed two generation semantics: independent per-arm generation (two independent LLM draws per task, one for baseline and one for guarded) or shared_initial_candidate (a single LLM draw per task whose candidate is evaluated by both arms). The independent per-arm mode is the stricter causal design for testing "adding a verifier" because it permits arm-level stochastic differences that can create discordant paired outcomes (n10 or n01). The executed run used shared_initial_candidate semantics to conserve hosted model budget; this choice and its exact accounting are recorded in execution/model_configuration.json and execution/execution_manifest.json within the archive. The archived model_configuration.json declares the requested model identifier and the execution semantics (declared_model_version = "gpt-5-mini", independent_condition_generation = false). For precise

accounting: executed model_calls_used = 200 while the preregistered maximum_model_calls allocated for independent per-arm generation (two draws per task) would have been up to 400. The model_configuration.json artifact (path: execution/model_configuration.json) has SHA-256 = 1acce92558edeb13cd97b75817329d332afe4a36d01d566f1a26c61bd072b4db7c8e29212ef28b97baccb66a507a7c8b8ceaad35b7e84f7976a9511cb072b4db7c8e29212ef28b97baccb66a507a7c8b8ceaad35b7e84f7976a951 and analysis/condition_summary.csv (path: analysis/condition_summary.csv; SHA-256 = 1cb072b4db7c8e29212ef28b97baccb66a507a7c8b8ceaad35b7e84f7976a951) - Claim: Deterministic reconciliation and absence of technical failures. Artifact: analysis/deterministic_reconciliation.json (path: analysis/deterministic_reconciliation.json; SHA-256 = 9d66242d13546c8819fbce6c38b6cb450d6454537a546fc49a7af3babb0c28e6b11b8b4ec6c873bb581f2dc5a42302f97ee3941868656ab8420546364f54000) - Claim: Representative per-task validator traces showing valid = true and violation_count = 0. Artifacts: execution/scoring/task-000001-baseline.json and execution/scoring/task-000001-guarded.json (examples archived under execution/scoring/; see artifact examples in bundle). The shared response files used by both arms are archived at execution/responses/*.txt with hashes included in the manifest.

Missingness, retry, contamination, and deterministic reconciliation checks. The preregistered missingness plan allowed one automatic retry per failed model call; pairs with unresolved technical failures after retry were to be excluded and replaced. Contamination detection was deterministic: prompt and response hashes were recorded and duplicate prompts would trigger exclusion + deterministic replacement. The execution produced zero technical failures and zero exclusions; deterministic_reconciliation.json (archived) records completed_episode_count = 400, complete_pair_count = 200, and provider audit counts (hosted_model_call_event_count = 200). The deterministic_reconciliation.json artifact is archived (SHA-256 = 9d66242d13546c8819fbce6c38b6cb450d6454537a546fc49a7af3babb0c28e6b11b8b4ec6c873bb581f2dc5a42302f97ee3941868656ab8420546364f54000) and documents that all planned consistency checks passed.

Analysis pipeline and concrete evidence links. The binary outcomes were computed from validator.valid flags and the paired 2×2 contingency counts were produced by paired_binary_analysis_v1. Key analysis artifacts and checks are archived with SHA-256 values: paired contingency (analysis/paired_contingency_table.csv, SHA-256 = 7bc1bd2a42b5ac3f7adb61db47cf8dbe0472f678032abcd1e7c44f92a2d57b11cb072b4db7c8e29212ef28b97baccb66a507a7c8b8ceaad35b7e84f7976a951) and the paired-difference figure (analysis/paired_difference_figure.svg, SHA-256 = ae5b8e0c644f8cf7579ff0b2daec77475310d4de96da6a4f1a9266a6bcb47211d) The exact McNemar p-value and CI reported in Results are taken directly from analysis/results.json produced by paired_binary_analysis_v1 and reconciled to the CSV artifacts above.

Evidence-to-claim mapping (concise, artifact-grounded). To reduce misinterpretation we map principal empirical/methodological claims in this paper to archival artifacts included in the bundle: - Claim: Executed generation semantics were shared_initial_candidate and model_calls_used = 200. Artifacts: execution/model_configuration.json (path: execution/model_configuration.json; SHA-256 = 1acce92558edeb13cd97b75817329d332afe4a36d01d566f1a26c61bd072b4db7c8e29212ef28b97baccb66a507a7c8b8ceaad35b7e84f7976a951) execution/raw_results.jsonl (path: execution/raw_results.jsonl; SHA-256 = 6b11b8b4ec6c873bb581f2dc5a42302f97ee3941868656ab8420546364f54000) - Claim: Confirmatory paired counts and SAFE rates (n11 = 200, baseline SAFE = 200/200, guarded SAFE =

200/200). Artifacts: analysis/paired_contingency_table.csv (path: analysis/paired_contingency_table.csv; SHA-256 = 7bc1bd2a42b5ac3f7adb61db47cf8dbe0472f678032abcd1e7c44f92a2d57b11cb072b4db7c8e29212ef28b97baccb66a507a7c8b8ceaad35b7e84f7976a951) and analysis/condition_summary.csv (path: analysis/condition_summary.csv; SHA-256 = 1cb072b4db7c8e29212ef28b97baccb66a507a7c8b8ceaad35b7e84f7976a951) - Claim: Deterministic reconciliation and absence of technical failures. Artifact: analysis/deterministic_reconciliation.json (path: analysis/deterministic_reconciliation.json; SHA-256 = 9d66242d13546c8819fbce6c38b6cb450d6454537a546fc49a7af3babb0c28e6b11b8b4ec6c873bb581f2dc5a42302f97ee3941868656ab8420546364f54000) - Claim: Representative per-task validator traces showing valid = true and violation_count = 0. Artifacts: execution/scoring/task-000001-baseline.json and execution/scoring/task-000001-guarded.json (examples archived under execution/scoring/; see artifact examples in bundle). The shared response files used by both arms are archived at execution/responses/*.txt with hashes included in the manifest.

Why shared_initial_candidate was permitted in preregistration. The preregistration allowed shared_initial_candidate semantics as a budget-conserving, planned option: it produces a single canonical candidate per task that both arms evaluate and reduces model calls when per-arm stochasticity is not required (e.g., initial feasibility or debugging runs). The preregistration allowed both semantics explicitly and documented the trade-off in the execution_contract. However, as explained in Results/Discussion, when the estimand is the causal effect of adding a validator to generation, independent per-arm generation is the necessary semantic to permit discordant paired outcomes; shared generation can collapse discordance and render McNemar-style inference vacuous. The archive documents both the allowed option and the executed choice (analysis/condition_summary.csv SHA above).

Operational reproducibility notes. All code, manifests, raw responses, scoring JSONs, and analysis artifacts in this section are archived in the supplied bundle under execution/ and analysis/ paths. The initial master prompt used to bootstrap the autonomous run is archived at provenance/master_prompt.txt (SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15) Replaying the confirmatory execution deterministically requires (1) execution/task_manifest.jsonl, (2) execution/responses/*.txt, (3) execution/scoring/*.json, (4) execution/model_configuration.json, and (5) the analysis executor paired_binary_analysis_v1 with the provided inputs; these are all present in the artifact bundle. The archived manifest and SHA values given above permit verification that the manuscript claims are supported by the exact artifacts used in analysis.

Summary of preregistered protections against post-hoc de-biasing. The preregistration included explicit exclusion rules (duplicate prompts, verifier nondeterminism), missingness/retry rules (1 retry), a stopping_rule (>10% technical failures or downgrade), and multiplicity controls (Bonferroni correction when reporting both benign and adversarial conditions). The execution adhered to these rules deterministically.

tically and recorded zero exclusions or technical failures (see `deterministic_reconciliation.json` SHA above).

IV. RESULTS

Execution and provider accounting (artifact pointers). The execution manifest records `completed_episode_count = 400` (200 paired episodes), `failed_episode_count = 0`, and `model_calls_used = 200` (see `execution/execution_manifest.json` and `execution/model_configuration.json` for declared model and generation semantics). `Deterministic_reconciliation.json` and `execution/execution_manifest.json` document that `hosted_model_call_event_count = 200`, `cache_miss_count = 200`, and that all deterministic consistency checks passed; no episodes were excluded or flagged (analysis/contamination_summary.csv reports zero flagged episodes) and analysis/missingness_summary.csv reports `complete_pairs = 200`.

Primary 2×2 paired outcomes and exact inference. The confirmatory contingency table derived by `paired_binary_analysis_v1` is: `n11 = 200` (SAFE in both arms), `n10 = 0` (SAFE guarded, UNSAFE baseline), `n01 = 0` (SAFE baseline, UNSAFE guarded), `n00 = 0` (UNSAFE both). These counts are archived in `analysis/paired_contingency_table.csv`. From these counts the point estimate for the paired SAFE-rate difference (guarded - baseline) equals 0.0. The preregistered exact McNemar two-sided test yields $p = 1.0$ under the observed contingency. The paired bootstrap percentile 95% confidence interval computed by the analysis executor (10000 resamples, seed = 1729) is degenerate at [0.0, 0.0]; the analysis log and bootstrap parameters are archived in `analysis/analysis_log.jsonl` and `analysis/paired_difference_figure.svg`.

Representative per-task diagnostics (scoring JSON evidence). Representative scoring artifacts (`execution/scoring/task-000001-baseline.json` and `task-000001-guarded.json`) demonstrate that the `shared_initial_candidate` content was identical across arms and that the validator trace fields `validation_before` and `validation_after` both report `valid = true` and `violation_count = 0`. Repair provider traces are null in all confirmatory scoring JSONs (`repair_provider_trace_path = null`), confirming that no automatic repair calls altered any candidate during confirmatory episodes. Representative `shared_initial` response files (`execution/responses/task-000001-shared-initial.txt`, `task-000002-shared-initial.txt`, `task-000003-shared-initial.txt`) contain the canonical configurations used for scoring; these files match the `normalized_response` fields in the scoring JSONs.

Detailed validation metadata. Each scoring JSON embeds validator metadata: `scorer_id = deterministic_netops_validator_v5`, `scorer_version = 5.0`, `scoring_rule = full_intent_constraint_validation`, and `scoring_status = COMPLETED`. Validator traces include `parsed_command_count`, `required_command_count`, `observed_touched_field_count`, and difficulty annotations (e.g.,

level = medium/hard/very_hard) for each task; these per-task difficulty tags are archived in `execution/task_manifest.jsonl` and appear in the representative task entries. The validator reported zero `violation_codes` and an empty `violations` array in every confirmatory episode’s `validation_after` block, consistent with the binary SAFE labels used for the paired analysis.

Why the confirmatory test was uninformative (artifact-grounded diagnosis). Three concrete archived facts explain the degenerate contingency: (1) the run executed `shared_initial_candidate` semantics (`independent_condition_generation = false` in `execution/model_configuration.json`) so both arms scored the same single LLM output per task; (2) the sampled gpt-5-mini outputs were labeled SAFE by `deterministic_netops_validator_v5` for every confirmatory task (baseline SAFE = 200/200 as recorded in `analysis/condition_summary.csv`); and (3) no repair calls were applied in confirmatory episodes (`repair_provider_trace` fields are null), so the guarded arm could not produce arm-specific candidate changes. Together these facts produce perfect concordance (`n11 = 200`) and therefore zero observed discordant pairs, which makes McNemar inference vacuous ($p = 1.0$) and yields a degenerate bootstrap CI.

Power assumptions vs observed data. The preregistered power calculation assumed a nonzero baseline unsafe rate (~10–15%) and discordant proportions approximated by $p10 = 0.07$ and $p01 = 0.01$ (preregistration text), yielding a required sample size ≈ 174 for 80% power to detect a ~7 percentage-point paired reduction. The executed shared semantics reduced model calls (200 used vs planned 400 for independent per-arm draws) and removed arm stochasticity; consequently the empirical baseline unsafe rate observed here (0%) contradicts the power assumptions and collapses inferential sensitivity. This is an execution-semantic and budgeting outcome rather than evidence about verifier effectiveness.

Exploratory reserves and uncomputed diagnostics. The preregistration reserved up to 50 exploratory tasks to estimate verifier ROC and refusal trade-offs; that reserve was not consumed before confirmatory execution and therefore no empirical ROC or refusal-rate estimates were produced in the archived confirmatory artifacts. `Analysis/contamination_summary.csv` and `analysis/missingness_summary.csv` document that no exploratory ROC artifacts were appended to the confirmatory results. The absence of ROC or refusal data is an archival fact in the supplied bundle and limits secondary claims about verifier true-positive/false-negative behavior.

Sensitivity and reproducibility checks. `Deterministic_reconciliation.json` shows consistent episode accounting (`complete_pair_count = 200`; `n_11 = 200`; `provider_call_audit.hosted_model_call_event_count = 200`). Analysis artifacts (`paired_contingency_table.csv`, `condition_summary.csv`, `analysis_log.jsonl`) are self-consistent and sufficient to reproduce the numeric inference reported here. All raw scoring JSONs, `shared_initial` responses, and

the `model_configuration/execution_manifest` artifacts are archived in the bundle and permit deterministic replay of the exact shared-generation execution semantics that produced the degenerate contingency.

V. DISCUSSION

We expand the diagnostic interpretation and practical implications of the observed non-informative (ceiling) outcome, grounding each statement in archived execution and analysis artifacts.

Root cause synthesis. The degenerate contingency ($n_{11}=200$, $n_{10}=n_{01}=n_{00}=0$) results from the conjunction of three archived facts: (1) generation semantics: the run executed `shared_initial_candidate` (`independent_condition_generation = false`) and therefore produced exactly one LLM candidate per task that both arms scored; (2) validator outcomes: `deterministic_netops_validator_v5` labeled every sampled candidate as valid (baseline SAFE = 200/200); and (3) absence of repairs: `repair_provider_trace` fields are null in confirmatory scoring JSONs, so the guarded arm did not alter any candidate. These archived facts (`execution/model_configuration.json`, `execution/execution_manifest.json`, `execution/scoring/*.json`, `analysis/paired_contingency_table.csv`) together make the paired causal contrast mechanically untestable because the arms had no opportunity to disagree.

Implications for confirmatory design. A paired estimand that attributes differences to the presence of a deterministic verifier requires per-arm variation that could produce discordant outcomes. Under shared generation semantics, discordance can only arise from repair-driven changes in the guarded arm or nondeterministic validator behavior; neither occurred here. Accordingly, the preregistered power calculation—which assumed a baseline unsafe rate $\approx 10\text{--}15\%$ and discordant proportions that yield detectability with $N \approx 174$ —no longer applied. The archived `model_call` accounting quantifies the specific resource trade-off: independent per-arm generation would have required ≈ 400 model calls (preregistered `maximum_model_calls`), whereas the executed shared semantics used 200 calls (`execution/execution_manifest.json`). When budgets force a choice, the estimand should drive budgeting: per-arm independence is necessary to test the causal effect of adding a validator to generation.

Practical recommendations for informative repeats (artifact-grounded). All recommendations follow directly from the preregistration and execution artifacts: - Use independent per-arm generation for confirmatory paired contrasts unless the guarded arm is capable of deterministic repairs that will reliably produce candidate changes. For this study size ($N=200$), independent generation requires roughly 200 additional model calls (400 total) compared with the executed shared run (200 `model_calls_used`). This arithmetic is visible in the preregistered `execution_contract` and the executed `manifest`. - Prior to running the confirmatory set, consume a preregistered exploratory reserve (we reserved up to 50 tasks) to calibrate adversarial transforms until the baseline unsafe rate matches the power assumptions. Archive

these tuning runs and fix the transform parameters before the confirmatory execution so the confirmatory assumptions remain preregistered and reproducible. - If repairs are part of the guarded pipeline, instrument, archive, and analyze `repair_provider_trace` artifacts: quantify the fraction of UNSAFE→SAFE conversions and check whether repairs introduce new violations. In our run repair traces were null; therefore repairs did not serve as a mechanism to induce discordance. - Implement a deterministic pilot check step that inspects the contingency counts on a small holdout before committing to the full confirmatory test. If the pilot produces a ceiling/floor contingency (no discordant pairs), abort and adapt along preregistered contingency-preserving paths (e.g., enable independent per-arm draws or strengthen adversarial transforms) rather than proceeding to a confirmatory test that will be uninformative.

Alternative estimands and complementary analyses. When budgets or policy preclude independent per-arm draws, consider changing the estimand to one that remains testable under shared semantics: (a) estimate the absolute unsafe-proposal rate of the LLM baseline (single-arm proportion) with appropriate CIs; (b) treat the guarded pipeline as an accept/reject gate and estimate refusal and post-repair SAFE rates (requires repairs and repair traces); or (c) treat the validator as a diagnostic tool and run exploratory ROC estimation (requires ground truth UNSAFE injections and the exploratory reserve). These alternative estimands shift the required resources but preserve inferential validity; the preregistration explicitly listed several `secondary_estimands` and exploratory items that map to these alternatives.

Threats to generalization and validator completeness. The deterministic validator used here encodes a formalized `full_intent_constraint_validation` for the synthetic task family and ran deterministically in the sandboxed environment; however, its completeness relative to production failure modes is limited. The synthetic tasks restrict fields to `admin/mtu/vlan` and single-AS topologies; consequently, zero unsafe detections here do not imply general safety in production multi-vendor or multi-AS contexts. Future work should validate validators against broader operational rule sets and consider human adjudication for ambiguous cases; such extensions must be preregistered to avoid post-hoc instrument selection.

Operational implications. From an operator perspective, strict guardrails can reduce risk but may also increase false refusals and operator workload. Our archived run did not produce refusals, so we cannot quantify that trade-off here; nevertheless, the preregistered exploratory ROC reserve was intended to measure precisely this tension and should be consumed in future repetitions. For deployment decisions, teams should balance the cost of additional model calls (for independent draws or exploratory calibration) against the operational risk of executing unverified changes; our artifacts make the per-task resource implications explicit and replayable.

Reproducibility and reuse. All artifacts supporting the above diagnosis and recommendations are archived (`execution/*` and `analysis/*`). They permit deterministic replay of the

executed semantics (shared_initial_candidate), verification of model_call accounting (model_calls_used = 200 vs planned 400), and inspection of representative scoring traces that expose why no arm-level discordance occurred. We encourage future repeats to adopt the pilot-check and exploratory calibration steps described above and to record the exact decision points in the preregistration so that execution semantics remain auditable and the confirmatory estimand remains testable.

VI. LIMITATIONS

- Synthetic tasks and scope: tasks were deterministic synthetic topologies produced by deterministic_netops_task_generator_v5 and limited to single-AS, multi-device setups; results may not generalize to production, multi-AS, or vendor-specific features.
- Generation semantics: the executed run used shared_initial_candidate (independent_condition_generation = false), producing identical per-pair generations and creating a ceiling effect when shared candidates were valid.
- Adversarial strength: the preregistered adversarial templates used in this run did not elicit unsafe outputs from gpt-5-mini on the sampled tasks; stronger adversarial cases or explicit injected unsafe examples are required to measure verifier effect.
- Single-model scope: only the declared model (gpt-5-mini) was evaluated; no multi-model generality is claimed.
- Verifier completeness: deterministic_netops_validator_v5 ran deterministically in synthetic sandboxed states; external validation of validator completeness against all real-world NetOps failure modes is not provided.
- Operational external validity: no production deployment, operator-in-the-loop workload measurement, nor multi-vendor field trials were performed; operator-cost trade-offs remain unquantified at scale.

VII. CONCLUSION

We executed a fully archived preregistered paired experiment comparing gpt-5-mini baseline generation and a deterministic verifier-augmented pipeline on 200 synthetic NetOps tasks. Under the executed shared_initial_candidate semantics and for the declared model, both arms produced zero verifier-detected unsafe proposals (paired difference = 0.0; exact McNemar $p = 1.0$; 95% CI = [0.0, 0.0]). Because identical per-pair generations were used and because the observed baseline unsafe rate was 0%, the preregistered causal hypothesis was not testable in this run. The scientific contribution is therefore methodological and reproducibility-centred: (1) a complete deterministic preregistered run with archived artifacts that permit exact replay; (2) an artifact-grounded diagnosis of why this execution was non-informative for verifier efficacy; and (3) concrete, archive-supported recommendations (independent per-arm generation budgeting, adversarial calibration, and exploratory ROC estimation) for informative repeats. The archived execution and analysis artifacts cited throughout (execution/* and analysis/*) permit deterministic replication

of the diagnosis and the run outputs and should be consulted prior to attempting repeats or production extrapolation.

DISCLOSURE STATEMENT

Disclosure Statement (CNSM Agentic AI Researcher track): Declared LLM: gpt-5-mini (execution recorded in execution/model_configuration.json). Orchestration/framework: hosted_netops_gvr_v1 adapter and deterministic experiment harness (execution/execution_manifest.json). Exact initial master prompt: archived at provenance/master_prompt.txt with immutable SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15. Topic constraints and autonomy boundary: the human Research Director selected the broad topic family (Generative AI and LLMs for NetOps) prior to the autonomy lock; after the lock the autonomous researcher performed literature review, preregistration (cand-2026-01-prereg-v1), experiment design, code generation, execution, analysis, manuscript drafting, autonomous peer review, and revisions. Preregistration and execution: the preregistration was frozen before execution; key preregistered elements (estimand, paired design N=200, task generator, allowed generation semantics, scoring rules, and stopping rules) are recorded in cand-2026-01-prereg-v1 and archived in the manuscript bundle. Generation semantics and model-call accounting (executed): independent_condition_generation = false (shared_initial_candidate semantics) producing a single initial LLM candidate per task shared across arms; model_calls_used = 200 while the preregistered maximum_model_calls allocated for independent per-arm generation = 400 (execution/execution_manifest.json and execution/model_configuration.json). Validator and scoring: deterministic_netops_validator_v5 performed deterministic sandboxed validation and encoded outcomes as binary labels (1 = SAFE, 0 = UNSAFE) under scoring_rule = full_intent_constraint_validation; archived scoring JSONs (execution/scoring/*_json) contain validator traces showing validation_before/after and violation lists. Analysis executor: paired_binary_analysis_v1 computed the paired contingency, exact McNemar two-sided test, and paired bootstrap CI (resamples = 10000, seed = 1729); analysis artifacts include analysis/paired_contingency_table.csv and analysis/condition_summary.csv. Deviations and restarts: no post-preregistration design changes were executed; the observed model call count reflects the executed shared_initial_candidate semantics. Reproducibility pointers (concise): execution/raw_results.jsonl, execution/task_manifest.jsonl, execution/model_configuration.json, execution/execution_manifest.json, analysis/paired_contingency_table.csv, analysis/condition_summary.csv, and analysis/paired_difference_figure.svg are archived in the supplied bundle and permit deterministic replays. Human editing: no manual human edits to technical content, analyses, or manuscript text occurred after the autonomy lock. Pre-lock development: infrastructure rehearsals occurred prior to the

lock but were excluded from final empirical evidence unless reproduced under the post-lock execution.

REFERENCES

- [1] <https://arxiv.org/abs/2605.12729>
- [2] <https://doi.org/10.2139/ssrn.7132138>
- [3] <https://doi.org/10.36227/techrxiv.177004277.71795055/v1>
- [4] <https://doi.org/10.36227/techrxiv.177006109.97957916/v1>
- [5] <https://doi.org/10.1109/ojcoms.2026.3680457>
- [6] Xianren Zhang, Xianfeng Tang, Hui Liu, Zongyu Wu, Qi He, Dongwon Lee, Suhang Wang, "Divide-Verify-Refine: Can LLMs Self-align with Complex Instructions?", 2025, 10.18653/v1/2025.findings-acl.709
- [7] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026